

物联网安全技术实验报告 1-2

班级	物联网 2202	学号	223428010210
专业	物联网工程	姓名	陈梓欣
实验表现成绩		实验成绩	
报告成绩			

一、实验题目：实验环境基础与替换密码

二、实验地点：410-3

三、实验目的：

- 1 理解并掌握 Python 运行环境和基本操作；
- 2 理解 Python 程序的基本结构和语法特征；
- 3 编程并实现替换算法：凯撒加密、仿射加密、维吉尼亚加密算法；

四、实验内容

（完成题目代码及运行结果并截图保存，在注释中含学号、姓名信息）

1. 求 1-12 月份英文缩写。

```
#223428010210 陈梓欣

#输入月份数字，输出月份英文缩写
months="JanFebMarAprMayJunJulAugSepOctNovDec"
n=input("请输入月份的数字(1-12):")
#子字符串的起始位置
pos=(int(n)-1) * 3
monthAbbrev=months[pos:pos+3]
print("第"+n+"月份的缩写为: "+monthAbbrev+".")
```

```
请输入月份的数字(1-12): 3
第3月份的缩写为: Mar.
```

2. 二元一次方程求解。

```
# 223428010210 陈梓欣

print("二元一次方程组求解")
# 输入两个方程的系数 a1, b1, c1 和 a2, b2, c2
a1, b1, c1 = eval(input("输入第一个方程的系数 a1, b1, c1: "))
a2, b2, c2 = eval(input("输入第二个方程的系数 a2, b2, c2: "))
# 计算行列式
D = a1 * b2 - a2 * b1
if D != 0:
    Dx = c1 * b2 - c2 * b1
    Dy = a1 * c2 - a2 * c1
    x = Dx / D
    y = Dy / D
    print("方程组的解为: x =", x, ", y =", y)
else:
    # 检查是否是无解还是无穷解
    if a1 * c2 == a2 * c1 and b1 * c2 == b2 * c1:
        print("方程组有无穷多组解")
    else:
        print("方程组无解")
```

二元一次方程组求解
输入第一个方程的系数 a1, b1, c1: 2,3,4
输入第二个方程的系数 a2, b2, c2: 3,5,1
方程组的解为: x = 17.0 , y = -10.0

3. 运用蒙特卡洛方法求 PI 值:

```
# 223428010210 陈梓欣

#蒙特卡洛方法求PI
from random import random
from math import sqrt
#DARTS 总点数, hits圆内点数量
DARTS = 2**20
hits = 0

for i in range(1,DARTS):
    #生成随机点
    x, y = random(), random()
    #计算点到圆心的距离
    dist = sqrt(x**2 + y**2)
    if dist <= 1.0:
        #圆内点hits+1
        hits = hits + 1
pi = 4 * (hits/DARTS)
print("Pi is %s" % pi)
```

Pi is 3.1411895751953125

4. 运用蒙特卡洛方法求 $\cos(x)$ 函数与 x 轴面积值

```
# 223428010210 陈梓欣

#求cos函数面积
#蒙特卡洛求sin
from random import random
from math import cos
import numpy as np

DARTS=2**20
hits=0

for i in range(1,DARTS):
    x,y=0.5*(np.pi)*random(),random()
    if(y<=cos(x)):
        hits=hits+1

s=np.pi*hits/DARTS
print(s)

2.0021085811386796
```

5. 凯撒密码

(1) Ceaser 密码加密算法实现(静态 k 值)

```
# 223428010210 陈梓欣

# Caesar密码加密算法 (静态k=3)
# 固定偏移量 k
k = 3
# 输入明文 (只加密小写字母)
plaintext = input("请输入需要加密的消息 (小写英文字母): ")
# 加密结果
ciphertext = ""
# 遍历每个字符
for ch in plaintext:
    if 'a' <= ch <= 'z':
        # ord(ch): 字符转ASCII码; ord('a')是97
        shifted = (ord(ch) - ord('a') + k) % 26
        # 加密后转为大写字母输出
        ciphertext += chr(ord('A') + shifted)
    else:
        # 非小写字母原样输出
        ciphertext += ch

# 输出密文
print("加密后的密文为:", ciphertext)

请输入需要加密的消息 (小写英文字母): hello world!
加密后的密文为: KHOOR ZRUOG!
```

(2) Ceaser 密码解密算法实现(静态 k 值)

```
# 223428010210 陈梓欣

# Caesar密码解密算法 (静态k=3)
# 固定偏移量 k (加密时是 +k, 解密时是 -k)
k = 3
# 输入密文 (默认密文是大写英文字母)
ciphertext = input("请输入需要解密的密文 (大写英文字母): ")
# 解密结果
plaintext = ""
# 遍历每个字符
for ch in ciphertext:
    if 'A' <= ch <= 'Z':
        # ord('A') 是 65, 把字符转成 0~25 范围索引
        shifted = (ord(ch) - ord('A') - k) % 26
        # 解密结果输出为小写字母
        plaintext += chr(ord('a') + shifted)
    else:
        # 非大写字母原样输出
        plaintext += ch

# 输出明文
print("解密后的原文为:", plaintext)

请输入需要解密的密文 (大写英文字母): KHOOR ZRUOG!
解密后的原文为: hello world!
```

6. 凯撒密码动态 K 值

(1) 简单替换密码加密算法(动态 k 值)

```
# 223428010210 陈梓欣

#动态加密密钥 k 凯撒加密算法
#获得需要加密的M
key = input("请加密的密钥(1-25):")
plaintext = input("请输入加密消息:")
#建立映射表
plaint_list = "abcdefghijklmnopqrstuvwxyz"
precipher_list = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
cipher_list = precipher_list[int(key):int(key)+26]
#输入信息
ciphertext = ""
if len(plaintext)>0:
    #对消息中的每个字母进行查询
    for x in plaintext:
        #获得位置索引
        index = plaint_list.find(x)
        #通过索引获得替换字母
        ciphertext = ciphertext+cipher_list[index]
    #输出密文结果
    if len(ciphertext) > 0:
        print("明文序列为:"+plaint_list)
        print("密文序列为:"+cipher_list)
        print("消息为:"+plaintext)
        print("密文为:"+ciphertext)
    else:
        print("输入错误")

请加密的密钥(1-25): 6
请输入加密消息: welcome
明文序列为:abcdefghijklmnopqrstuvwxyz
密文序列为:GHIJKLMNOPQRSTUVWXYZABCDEF
消息为:welcome
密文为:CKRIUSK
```

(2) 简单替换密码解密算法(动态 k 值)

```
# 223428010210 陈梓欣

#动态加密密钥 k 凯撒解密算法
#获得需要加密的M
key = input("请解密的密钥(1-25):")
ciphertext = input("请输入解密消息:")
#建立映射表
plaint_list = "abcdefghijklmnopqrstuvwxyz"
precipher_list = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
cipher_list = precipher_list[int(key):int(key)+26]
#输入信息
plaintext = ""
if len(ciphertext)>0:
    #对消息中的每个字母进行查询
    for x in ciphertext:
        #获得位置索引
        index = cipher_list.find(x)
        #通过索引获得替换字母
        plaintext = plaintext+plaint_list[index]
    #输出密文结果
    if len(ciphertext) > 0:
        print("明文序列为:"+plaint_list)
        print("密文序列为:"+cipher_list)
        print("解密密文为:"+ciphertext)
        print("解密结果为:"+plaintext)
    else:
        print("输入错误")

请解密的密钥(1-25): 6
请输入解密消息: CKRIUSK
明文序列为:abcdefghijklmnopqrstuvwxyz
密文序列为:GHIJKLMNOPQRSTUVWXYZABCDEF
解密密文为:CKRIUSK
解密结果为:welcome
```

7. 仿射密码

(1) 仿射加密算法

```
# 223428010210 陈梓欣

# 仿射加密算法实现
# 加密公式:  $E(x) = (a*x + k) \bmod 26$ 
# 输入密钥a和k (注意: a必须与26互质)
a, k = eval(input("请输入加密的密钥 a, k (1~25): "))
# 明文输入
plaintext = input("请输入要加密的消息 (小写英文字母): ")
# 字母映射表
plaint_list = "abcdefghijklmnopqrstuvwxyz"
cipher_list = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
# 加密结果
ciphertext = ""
# 遍历明文字符
for ch in plaintext:
    if ch in plaint_list:
        index = plaint_list.find(ch)          # 找到字母索引
        pos = (a * index + k) % 26             # 加密变换
        ciphertext += cipher_list[pos]         # 加密字母 (大写)
    else:
        ciphertext += ch                       # 非字母直接加入
# 输出加密结果
print("原消息为:", plaintext)
print("加密后的密文为:", ciphertext)
```

```
请输入加密的密钥 a, k (1~25): 5,8
请输入要加密的消息 (小写英文字母): welcome
原消息为: welcome
加密后的密文为: OCLSAQC
```

(2) 仿射解密算法

```
# 223428010210 陈梓欣

# 仿射解密算法实现
# 解密公式:  $D(y) = a_{inv} * (y - k) \bmod 26$ 
def mod_inverse(a, m):
    """
    求 a 关于 m 的模逆元, 即  $a * a_{inv} \equiv 1 \pmod m$ 
    使用扩展欧几里得算法
    """
    for x in range(1, m):
        if (a * x) % m == 1:
            return x
    return None # 如果没有逆元, 返回 None
# 输入密钥 a 和 k
a, k = eval(input("请输入解密密钥 a, k (1~25): "))
# 求 a 的模 26 逆元
a_inv = mod_inverse(a, 26)
if a_inv is None:
    print("错误: 密钥 a 与 26 不互质, 无法解密!")
    exit()
# 输入密文
ciphertext = input("请输入要解密的密文 (大写英文字母): ")
# 字母表
cipher_list = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
plain_list = "abcdefghijklmnopqrstuvwxyz"
# 解密结果
plaintext = ""
# 遍历密文字符
for ch in ciphertext:
    if ch in cipher_list:
        index = cipher_list.find(ch)          # 找到密文字母索引 (0~25)
        pos = (a_inv * (index - k)) % 26       # 解密计算
        plaintext += plain_list[pos]           # 对应小写明文字母
    else:
        plaintext += ch                       # 非字母直接加入
# 输出明文
print("密文为:", ciphertext)
print("解密后的原文为:", plaintext)
```

```
请输入解密密钥 a, k (1~25): 5,8
请输入要解密的密文 (大写英文字母): OCLSAQC
密文为: OCLSAQC
解密后的原文为: welcome
```

8. 维吉尼亚加密算法

```
# 223428010210 陈梓欣

# 维吉尼亚加密算法 Vigenère Cipher
# 检查输入是否为全字母
def input_chk(message):
    chk_list = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
    for x in message.upper():
        if x not in chk_list:
            return False
    return True

# 凯撒加密 (单字符)
def caesar_encrypt_char(key, ch):
    plaint_list = "abcdefghijklmnopqrstuvwxyz"
    cipher_list = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    index = plaint_list.find(ch)
    if index == -1:
        return ch # 非字母直接返回
    return cipher_list[(key + index) % 26]

# 维吉尼亚加密
def vigenere_encrypt(key, message):
    strlist = "abcdefghijklmnopqrstuvwxyz"
    key = key.lower()
    message = message.lower()
    cipher = ""

    key_len = len(key)
    for i in range(len(message)):
        k = key[i % key_len] # 循环使用密钥
        key_index = strlist.find(k)
        m_char = message[i]
        cipher_char = caesar_encrypt_char(key_index, m_char)
        cipher += cipher_char
    return cipher

# 主函数
def main():
    key = input("请输入加密密钥 (字母): ")
    message = input("请输入加密消息 (字母): ")

    if not (input_chk(key) and input_chk(message)):
        print("输入错误: 密钥和消息必须只包含英文字母")
        return

    cipher = vigenere_encrypt(key, message)
    print("\n=====")
    print("明文消息: " + message)
    print("密钥: " + key)
    print("加密结果: " + cipher)
    print("=====")

if __name__ == "__main__":
    main()
```

```
请输入加密密钥 (字母): KEY
请输入加密消息 (字母): attackatdawn
```

```
=====
明文消息: attackatdawn
密钥: KEY
加密结果: KXRKGIXKBKAL
```

五、实验中遇到的问题及解决方法

问题：仿射加密算法中，加密不可逆的。

解决方法：修改输入的 a 值，使其与 26 互质。

问题：维吉尼亚加密算法运行时无法输出

解决方法：修改 key.find(x) 方法为直接使用 key[i % len(key)] 循环使用密钥，

并不再分段处理 `message`，改为逐字符按位加密。

六、实验总结及心得

在本次实验中，我通过编写并实现了多种经典加密算法，包括仿射加密与解密、凯撒加密与解密、维吉尼亚加密等，进一步加深了对古典密码学基本原理的理解。

通过编码实践，我认识到各类加密算法本质上都是基于字符位置变换的数学操作，例如仿射加密利用了模运算，凯撒加密是其特例，而维吉尼亚加密通过多字符密钥实现变换的多样性。此外，我还掌握了如何在程序中处理输入验证、大小写统一、密钥循环等细节问题，同时在调试过程中也发现了一些常见逻辑错误，如索引计算、变量作用域、切片边界等问题。

此次实验不仅锻炼了我的 `Python` 编程能力，也让我深刻体会到加密算法在信息安全中的重要作用。未来我希望能进一步探索现代密码算法及其在网络安全中的实际应用。